# Lab Exercise 5: Python and Pandas

Due: March 4, 2026 on Canvas

The purpose of this exercise is to practice working with dataframes. Feel free to work in your project groups! If you work together, please only submit one assignment per team.

1. Start a new Jupyter notebook and include the following packages:
```
import pandas as pd
import sqlite3 as sql
from matplotlib import pyplot as plt
import seaborn as sns
import numpy as np
```

2. Import the same `python.db` database from Canvas as for Lab 4. We will be using the `fastfood` table as before.
3. Load the data into a dataframe called `ff_df`.
4. Notice that the ZIP codes listed are not integers. Convert them to 5-digit integers using the code `ff_df["postalCodeNew"] = ff_df["postalCode"].map(lambda x: int(x[:5]))`. In a markup cell, explain briefly what the code is doing.
5. Which restaurant chain has the greatest representation in San Francisco, CA? (This city covers the ZIP Codes 94100-94188.)

In Pandas, you can use `.loc` and `.iloc` to access specific subsets of your dataframe. The big difference between the two is that `.loc` uses row and column names as the index whereas `.iloc` uses row and column *numbers* (starting at zero) as the index.

6. `.loc` practice
    a. Display the restaurant with the index 42 using `ff_df.loc[42]`. (If it were not numerical, you could access it using `ff_df.loc["restaurant_name"]`, for example.
    b. Display restaurants 42-45 using `ff_df.loc[42:45]`.
    c. Display restaurants 24 and 42 using `ff_df.loc[[24, 42]]`. (Notice `[24,42]` has to be passed as a list.)
    d. List all restaurants in Oakland using `ff_df.loc[ff_df["city"]=="Oakland"]`. (Notice how we are conditioning the rows based on the *city column* having a certain value!)
    e. You can use `.loc` to also select specific columns. This is similar to `SELECT COLUMN_NAME FROM TABLE` in SQL. Display only the restaurant's name and city

using `ff_df.loc[:, ["name", "city"]]`. Notice again the use of a list to pass in two parameters. If only returning a single column, you do not need to use a list.

f. Display the name and city for only Hawaii locations using `ff_df.loc[ff_df["province"] == "HI", ["name", "city"]]`.

g. Make up two sets of data on your own that you would like to return and determine the code necessary to do so. Then do so.

7. `.iloc` practice:
   a. Display record 42 using `ff_df.iloc[42]`. (Note that `.iloc` starts counting at 0.)
   b. Repeat 6e using `.iloc`. The syntax is the same except that you are using column numbers instead of names.
   c. Display the restaurant name and city for the 1st and 42nd rows (not record numbers 1 and 42!) using `.iloc`.

8. Use the following code to generate a list of restaurants that have at least 50 locations:

```
counts = ff_df['name'].value_counts()
high_counts = counts[counts >= 50]
```

   Then use the following code to create a bar chart of these restaurants. Ignore the error for now.

```
_, ax = plt.subplots(figsize=(15, 6))

ax.bar(x=high_counts.index, height=high_counts)
ax.set_xticklabels(labels=high_counts.index, rotation=80);
```

9. Show a scatterplot of the restaurants around Buffalo, NY:

```
buffalo = ff_df[ff_df['city'] == 'Buffalo']

_, ax = plt.subplots(figsize=(8, 8))

sns.scatterplot(data=buffalo,
                x=buffalo['longitude'],
                y=buffalo['latitude'],
                hue=buffalo['name'],
                ax=ax);
```

10. Repeat the exercise in #9 with a city of choosing!